DP-OPS-0031: Pointer-First Agent Constitution & System Hardening

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/agent-constitution-0031

Base HEAD: 343c3dc95

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the Required Work Branch listed above (no branch creation or switching). STOP if any mismatch. STOP if the Required Work Branch is missing. STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - docs/library/agents/R-AGENT-01.md
  - docs/library/agents/R-AGENT-02.md
  - docs/library/agents/R-AGENT-03.md
  - docs/library/agents/R-AGENT-04.md
  - docs/library/agents/R-AGENT-05.md
  - docs/library/agents/R-AGENT-06.md
  - ops/bin/llms
  - ops/lib/scripts/agent.sh
  - llms.txt

(OPEN is for Integrator/DP Writer state refresh;

Worker does not read OPEN). DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Finalize the "Pointer-First" agent constitution by removing legacy metadata, incorporating llms.txt into the automated context system, and hardening the agent lifecycle script against agency drift to establish a Proof-of-Product for the Stela Governance Architecture.
- **Non-Goals (optional, drift-killer):**
  - No changes to projects/ payloads.
  - No changes to PoT.md or GOVERNANCE.md (Constitution is fixed; we are aligning implementation).
  - No introduction of external "Stella" or "Strella" agent frameworks; this work is strictly internal to the Stela repository constitution.[1]

Target Files allowlist (hard gate)

- docs/library/agents/R-AGENT-01.md
- docs/library/agents/R-AGENT-02.md
- docs/library/agents/R-AGENT-03.md
- docs/library/agents/R-AGENT-04.md
- docs/library/agents/R-AGENT-05.md
- docs/library/agents/R-AGENT-06.md
- ops/bin/llms
- ops/lib/scripts/agent.sh
- llms.txt
- SoP.md

Allowlist rule: exact paths only;

use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**

    - Workers do not create/switch branches.
- Workers do not stage/commit/merge/push.
    - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**.
  If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
    - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
- STOP if required DP inputs are missing (do not guess).
- STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

1. Theoretical Framework: The Pointer-First Architecture

The current state of the Stela repository reflects a transitional phase in the evolution of Artificial Intelligence governance. We are moving from an "Embedded Constitution" model—where agents contain their own instructions, prompts, and tool definitions—to a "Pointer-First Constitution." This shift is not merely syntactical but fundamental to the preservation of truth in a non-deterministic system.

In traditional computing, particularly within languages like C++ or Rust, the concept of a pointer allows a system to reference a memory address rather than duplicating the data contained therein.[5] This ensures that when the data at the source changes, all references to that data reflect the change instantaneously. There is no risk of "stale data" because the consumer does not hold the data; it holds the address.

In the context of Large Language Models (LLMs) and agentic workflows, this principle has historically been ignored. Agents are typically defined with massive "System Prompts" that copy-paste rules, behavioral standards, and project context directly into the agent's definition.[6] This creates a "Split Brain" problem: if the global governance (e.g., PoT.md) changes, the agent's local definition remains static, leading to governance drift.

The "Pointer-First" architecture proposed for Stela [7] mandates that agents must not contain "Canon Text." Instead, they must contain "Pointers" to the Canonical Surfaces (PoT.md, GOVERNANCE.md). When an agent is rehydrated, it follows these pointers to load the *current* truth, rather than relying on a cached and potentially obsolete version of the truth. This mirrors the "Pointer Network" architectures in neural machine learning where output tokens reference input positions rather than generating new vocabulary [7], thereby increasing fidelity and reducing hallucination.

The current implementation of agents R-AGENT-01 through R-AGENT-06 violates this

doctrine. They contain legacy YAML metadata headers (e.g., tools:..., model:..., description:...) which are essentially embedded configuration data. This data is "dead" because it is not validated against the ops/ runtime capabilities. By refactoring these agents to a pure Markdown structure that *points* to their authorized toolchains and governance constraints, we align the repository with the "Pointer-First" doctrine.

2. The Dispatch Packet as Governance Kernel

The Stela system utilizes the "Dispatch Packet" (DP) as its atomic unit of work. This concept is derived from heterogeneous computing architectures (like HSA or CUDA), where a "kernel dispatch packet" contains all the state, dimension, and synchronization information required for a GPU to execute a specific task without further host intervention.[9]

In the Stela context, the DP serves a similar function for the Agentic Workforce. It is a self-contained, immutable contract that defines:

1. **State:** The exact git hash and context required.
2. **Logic:** The specific transformation to apply.
3. **Boundary:** The hard limits (allowlists) of the operation.

Current analysis of the repository reveals that while the DP structure is defined in TASK.md, the *enforcement* of this structure is partially manual. The integration of llms.txt—the primary discovery map for these packets—is currently static. This means that as new Dispatch Packets are architected or new agents are onboarded, the discovery layer does not automatically update. This introduces latency and potential error in the "Dispatch" phase, similar to a packet processor failing to update its queue tail pointer.[11]

To remedy this, ops/bin/llms must be upgraded to dynamically generate llms.txt based on the actual file state of the repository. This ensures that the "Map" (llms.txt) is always a distinct reflection of the "Territory" (ops/, docs/), adhering to the "Flat Truth" visibility principle.[13]

3. Agency Guardrails and the Risk of Drift

The most critical vector of risk in an agentic repository is "Agency Drift." This occurs when an agent, designed for a specific narrow task (e.g., "Documentation Update"), gains the capability or the "hallucinated permission" to execute broader tasks (e.g., "Architecture Refactor").

Research into AI safety frameworks indicates that "Agency" is not a binary state but a spectrum that requires strict "Guardrails".[16] Without these guardrails, agents may engage in "snowballing" behaviors where a small authorization leads to a cascade of unauthorized tool use.[18] In a repository where agents have write access (via the Operator), this is equivalent to a "Harvest Now, Decrypt Later" vulnerability [19], where an agent might harvest credentials or context it should not have access to.

The current ops/lib/scripts/agent.sh script facilitates the "Harvest" and "Promote" lifecycle but lacks a "Check" mechanism to enforce these guardrails. It does not verify if an agent has defined a ## Scope Boundary. It does not strictly check for "Context Hazards" (e.g., an agent including the agent library in its own context, leading to recursive logic loops).

To establish a true "Proof-of-Product" [20] for the Stela governance model, we must harden agent.sh to function as a "Security Gate".[22] It must strictly enforce that every agent has a defined boundary and that no agent can access the "God Mode" context of the full library unless explicitly authorized. This aligns with the "Zero Trust Governance" models proposed for high-maturity agentic frameworks.[17]

4. Summary of Drifts

- **Drift 1 (Ontological):** Agents R-AGENT-01 to 06 use legacy YAML metadata, violating the "Pointer-First" linguistic standard.
- **Drift 2 (Operational):** llms.txt is static and manual, violating the "Automation First" principle of the Ops Kernel.
- **Drift 3 (Security):** agent.sh lacks active guardrail enforcement (check command), leaving the system vulnerable to agency drift.

5. Desired State

The desired state is a **Hardened, Pointer-First Repository** where:

1. Agents are pure prose documents that point to PoT.md for their laws.
2. Discovery (llms.txt) is an automated downstream artifact of the repository state.
3. The Agent Lifecycle (agent.sh) includes a mandatory audit step (check) that enforces scope boundaries and prevents context hazards before an agent can be promoted or deployed.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Refactor Agent Constitution (Pointer-First Transition):**
   The worker shall rewrite docs/library/agents/R-AGENT-01.md through R-AGENT-06.md. This refactor is not merely cosmetic; it is a constitutional amendment to the files.
   - **Action:** Remove all YAML frontmatter (the text between --- and --- at the top of the files).
   - **Action:** Relocate the name and description data into the ## Role section. The Role description must be expanded to full sentences, avoiding contractions.
   - **Action:** Relocate tools and model data into the ## Specialization section.
   - **Action:** Ensure every agent contains a ## Pointers section that explicitly links to PoT.md, docs/GOVERNANCE.md, and TASK.md. This establishes the "Pointer-First" authority.
   - **Action:** Ensure every agent contains a ## Scope Boundary section. This is the

primary "Agency Guardrail".[16] It must explicitly state what the agent *cannot* do or where its jurisdiction ends.
    - **Reference:** Use the exact markdown structures provided in the "Patch / Diff" section of this DP.
2. **Automate Context Discovery (LLMS Upgrade):**
   The worker shall modify ops/bin/llms to programmatically generate llms.txt.
    - **Theory:** llms.txt is the "robots.txt" for AI agents.[14] It must be accurate. A manual file is a dead file.
    - **Action:** Append logic to ops/bin/llms that writes llms.txt to the repository root.
    - **Logic:** The script must iterate through the defined context bundles (Small and Full) and the operational binary list, formatting them into a standard Markdown list.[15]
    - **Constraint:** The generated file must include headers for "Governance & Jurisdictions," "State & Continuity," "Operations & Manuals," and "Operational Binaries."
3. **Harden Agent Lifecycle (Guardrail Implementation):**
   The worker shall upgrade ops/lib/scripts/agent.sh to include a check command.
    - **Theory:** Trust but Verify. We cannot assume agents will respect boundaries; we must enforce them via code.[17]
    - **Action:** Implement cmd_check_agency().
    - **Logic 1 (Legacy Purge):** The check must scan all agents in docs/library/agents/ and fail if any YAML headers (^---$) are detected. This enforces the "No Legacy Metadata" rule.
    - **Logic 2 (Boundary Enforcement):** The check must fail if an agent file lacks a ## Scope Boundary section. This enforces the "Agency Guardrail."
    - **Logic 3 (Hazard Detection):** The check must fail if an agent contains self-referential links to the agent library that would cause a context loop (e.g., an agent loading docs/library/agents/).
    - **Integration:** Register this new command in the script's main case switch.
4. **Verify and Validate (Proof-of-Product):**
   The worker shall execute the newly created tools to prove the system works.
    - **Action:** Run ./ops/bin/llms and verify llms.txt is updated.
    - **Action:** Run bash ops/lib/scripts/agent.sh check and verify it passes (indicating all agents are correctly refactored and hardened).
    - **Action:** Capture these results in the DP-OPS-0031-RESULTS.md receipt.

Required Content:

The explicit content for the refactored agents is strictly defined in Section 3.4.4. The worker must not invent new roles or change the "Provenance" data (which is historical fact).

3.4.3 Changelog

- **Refactor:** Converted R-AGENT-01 through R-AGENT-06 from legacy YAML-hybrid format

to the Stela "Pointer-First" Markdown standard, eliminating embedded configuration drift.
- **Feat:** Upgraded ops/bin/llms to automatically generate llms.txt, ensuring the AI discovery layer is perfectly synchronized with the repository state.
- **Security:** Hardened ops/lib/scripts/agent.sh with a new check command that enforces "Agency Guardrails" (Scope Boundaries) and detects "Context Hazards," establishing a zero-trust architecture for agent promotion.
- **Docs:** Updated SoP.md to record the governance refactor and the establishment of the new agent constitution.

3.4.4 Patch / Diff

1. ops/bin/llms (Update to generate llms.txt)

This patch implements the "Automated Discovery" requirement. By generating llms.txt at runtime, we ensure that external agents always see the current state of the "Map," preventing the "Dead Pointer" phenomenon described in.[13]

Bash

```
<<<<
# existing content...
printf 'Wrote %s\n' "${written[*]}"
====
#... (after existing content)

# Generate llms.txt (Discovery Map)
LLMS_TXT="${ROOT_OUT_DIR}/llms.txt"

{
  echo "# Stela llms.txt"
  echo "# Pointer map to canonical docs and tools (no duplication)."

  echo "## 0. Context Bundles"
  echo "- llms-small.txt - Small context bundle (PoT, SoP head 10, docs/library/INDEX.md)."
  echo "- llms-full.txt - Full context bundle (small + README + docs/library)."

  echo "## 1. Governance & Jurisdictions"
  echo "- PoT.md - Policy of Truth: constitution, staffing, jurisdiction, and enforcement."
  echo "- docs/GOVERNANCE.md - Non-negotiables and project coherence rules."

  echo "## 2. State & Continuity"
```

```
  echo "- SoP.md - Historical ledger (State of Play)."
  echo "- TASK.md - Active work surface, DP contract, and thread log."
  echo "- docs/library/MAP.md - Wayfinding for cross-session state."
  echo "- ops/lib/manifests/CONTEXT.md - Required context artifact checklist."

  echo "## 3. Operations & Manuals"
  echo "- docs/library/MANUAL.md - Command mechanics and cheat sheet."
  echo "- docs/library/INDEX.md - Curated index of approved operator surfaces."
  echo "- docs/library/SKILLS.md - Skills promotion template and packet ledger."
  echo "- docs/INDEX.md - Documentation front door."

  echo "## 4. Operational Binaries"
  echo "- ops/bin/open - Prompt generation and freshness gate."
  echo "- ops/bin/dump - State capture for agent ingestion."
  echo "- ops/bin/help - Context-aware documentation search."
} > "$LLMS_TXT"

written+=("$LLMS_TXT")
printf 'Wrote %s\n' "${written[*]}"
>>>>
```

2. ops/lib/scripts/agent.sh (Harden)

This patch introduces the check command. This acts as the "Guardrail" logic.[16] By enforcing the absence of YAML and the presence of Scope Boundary, we mathematically prove that the agents adhere to the constitution before they are allowed to operate.

Bash

```
<<<<
#... inside main switch case...
  promote|--promote)
    shift
    cmd_promote "$@"
    ;;
  check|--check)
    shift
    cmd_check_agency "$@"
    ;;
#...
```

```
====
#... (Add new function)
cmd_check_agency() {
 require_repo_root

 local failed=0
 echo "Stela Agency Guardrail Audit"
 echo "----------------------------"

 # 1. Check for Legacy YAML Headers
 # We grep for the separator. If found, it indicates legacy formatting.
 local agents_with_yaml
 agents_with_yaml=$(grep -l "^---$" "$AGENTS_DIR"/*.md 2>/dev/null |

| true)
 if [[ -n "$agents_with_yaml" ]]; then
  echo "FAIL: Legacy YAML headers detected in agents:"
  echo "$agents_with_yaml"
  failed=1
 else
  echo "PASS: No legacy YAML headers found."
 fi

 # 2. Check for Scope Boundary
 # Every agent must define what it CANNOT do.
 local agent
 for agent in "$AGENTS_DIR"/*.md; do
  if [[! -e "$agent" ]]; then continue; fi
  if! grep -q "^## Scope Boundary" "$agent"; then
   echo "FAIL: Missing '## Scope Boundary' in $(basename "$agent")"
   failed=1
  fi
 done
 if [[ $failed -eq 0 ]]; then
  echo "PASS: All agents define Scope Boundaries."
 fi

 # 3. Context Hazard Check
 # Agents cannot load the agent library. This prevents recursive definition loops.
 local hazard_pattern="docs/library/agents"
 local hazards
 hazards=$(grep -l "$hazard_pattern" "$AGENTS_DIR"/*.md 2>/dev/null |
```

```
| true)
  if [[ -n "$hazards" ]]; then
    # We allow the Pointers section to mention the file itself or siblings if strictly necessary,
    # but generally, an agent shouldn't be loading the library.
    # For this strict check, we warn if they try to ingest the directory.
    echo "WARN: Potential Context Hazard (reference to agents dir) in:"
    echo "$hazards"
    # We don't fail yet as some might validly reference the registry, but we flag it.
  fi

  if [[ $failed -eq 1 ]]; then
    echo "Audit FAILED."
    exit 1
  fi
  echo "Audit PASSED."
}
>>>>
```

3. docs/library/agents/R-AGENT-01.md (Refactor)

This file is refactored to remove the name: architect YAML and place it into the Role section. The Scope Boundary is explicitly defined to prevent architectural overreach.

Agent: architect

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f8917d395b6b027710385cd647ae179850b
- **Objective:** Agent System Upgrade (Pointer-First Constitution + Provenance Lifecycle)

Role

Software architecture specialist for system design, scalability, and technical decision-making.

Use PROACTIVELY when planning new features, refactoring large systems, or making architectural decisions.

Specialization

Architecture planning and system design.

- **Model:** opus
- **Tools:** Read, Grep, Glob

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump, ops/bin/llms, tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh
- JIT skills:
  - docs/library/skills/S-LEARN-02.md
  - docs/library/skills/S-LEARN-03.md
  - docs/library/skills/S-LEARN-06.md

Scope Boundary

Operate only within the active DP scope and defer to canon surfaces for governance and behavioral rules.

4. docs/library/agents/R-AGENT-02.md (Refactor)

Agent: code-reviewer

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f8917d395b6b027710385cd647ae179850b
- **Objective:** Agent System Upgrade (Pointer-First Constitution + Provenance Lifecycle)

Role

Expert code review specialist. Proactively reviews code for quality, security, and maintainability.

Use immediately after writing or modifying code. MUST BE USED for all code changes.

Specialization

Code review for quality, security, and maintainability.

- **Model:** opus

- **Tools:** Read, Grep, Glob, Bash

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump, ops/bin/llms, tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh
- JIT skills:
  - docs/library/skills/S-LEARN-01.md
  - docs/library/skills/S-LEARN-04.md
  - docs/library/skills/S-LEARN-05.md

Scope Boundary

Operate only within the active DP scope and defer to canon surfaces for governance and behavioral rules.

5. docs/library/agents/R-AGENT-03.md (Refactor)

Agent: doc-updater

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f8917d395b6b027710385cd647ae179850b
- **Objective:** Agent System Upgrade (Pointer-First Constitution + Provenance Lifecycle)

Role

Documentation and codemap specialist. Use PROACTIVELY for updating codemaps and documentation.

Runs /update-codemaps and /update-docs, generates docs/CODEMAPS/*, updates READMEs and guides.

Specialization

Documentation and codemap maintenance.

- **Model:** opus

- **Tools:** Read, Write, Edit, Bash, Grep, Glob

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump, ops/bin/llms, tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh
- JIT skills:
  - docs/library/skills/S-LEARN-01.md
  - docs/library/skills/S-LEARN-06.md
  - docs/library/skills/S-LEARN-08.md

Scope Boundary

Operate only within the active DP scope and defer to canon surfaces for governance and behavioral rules.

6. docs/library/agents/R-AGENT-04.md (Refactor)

Agent: integrator

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f8917d395b6b027710385cd647ae179850b
- **Objective:** Agent System Upgrade (Pointer-First Constitution + Provenance Lifecycle)

Role

Expert planning specialist for complex features and refactoring.

Use PROACTIVELY when users request feature implementation, architectural changes, or complex refactoring. Automatically activated for planning tasks.

Specialization

Planning and integration for complex changes.

- **Model:** opus
- **Tools:** Read, Grep, Glob

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump, ops/bin/llms, tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh
- JIT skills:
  - docs/library/skills/S-LEARN-06.md
  - docs/library/skills/S-LEARN-07.md
  - docs/library/skills/S-LEARN-08.md

Scope Boundary

Operate only within the active DP scope and defer to canon surfaces for governance and behavioral rules.

7. docs/library/agents/R-AGENT-05.md (Refactor)

Agent: refactor-cleaner

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f8917d395b6b027710385cd647ae179850b
- **Objective:** Agent System Upgrade (Pointer-First Constitution + Provenance Lifecycle)

Role

Dead code cleanup and consolidation specialist.

Use PROACTIVELY for removing unused code, duplicates, and refactoring. Runs analysis tools (knip, depcheck, ts-prune) to identify dead code and safely removes it.

Specialization

Refactor and dead-code cleanup.

- **Model:** opus
- **Tools:** Read, Write, Edit, Bash, Grep, Glob

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump, ops/bin/llms, tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh
- JIT skills:
  - docs/library/skills/S-LEARN-01.md
  - docs/library/skills/S-LEARN-04.md
  - docs/library/skills/S-LEARN-06.md

Scope Boundary

Identify and remove dead code or duplication while preserving behavior within the active DP scope.

8. docs/library/agents/R-AGENT-06.md (Refactor)

Agent: security-reviewer

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f8917d395b6b027710385cd647ae179850b
- **Objective:** Agent System Upgrade (Pointer-First Constitution + Provenance Lifecycle)

Role

Security vulnerability detection and remediation specialist.

Use PROACTIVELY after writing code that handles user input, authentication, API endpoints, or sensitive data.

Flags secrets, SSRF, injection, unsafe crypto, and OWASP Top 10 vulnerabilities.

Specialization

Security review and vulnerability detection.

- **Model:** opus

- **Tools:** Read, Write, Edit, Bash, Grep, Glob

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump, ops/bin/llms, tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, tools/verify.sh
- JIT skills:
  - docs/library/skills/S-LEARN-01.md
  - docs/library/skills/S-LEARN-04.md
  - docs/library/skills/S-LEARN-05.md

Scope Boundary

Operate only within the active DP scope and defer to canon surfaces for governance and behavioral rules.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- bash tools/lint/context.sh
- bash tools/lint/truth.sh
- bash ops/lib/scripts/agent.sh check (NEW: Validate agency guardrails)
- ./ops/bin/llms (NEW: Validate llms.txt generation)
- cat llms.txt (Visually inspect freshness)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.

- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0031-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-work-agent-constitution-0031-*.txt (required)
- storage/handoff/OPEN-work-agent-constitution-0031-*.txt (required)
- storage/dumps/dump-platform-...
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to

storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0031-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

5. Work Log (Timestamped Continuity)

> **NOTICE:** This log is for the *active* DP only.
>
> Historical logs are archived in 'storage/archives/'. Do not rely on this log for long-term history.
>
> **Rule:** This is not a transcript. It is the durable breadcrumb trail.
>
> **Expectation:** Append a new timestamped entry on every DP closeout (no exceptions).
>
> **Each entry ends with:** NEXT: <one single action>.

- *2026-02-06 23:15* — DP-OPS-0031: Pointer-First agent constitution initiated. Plan drafted including llms.txt automation and agent.sh hardening. Verification: NOT RUN (Drafting phase). Blockers: none. NEXT: Worker execution of DP-OPS-0031.

Works cited

1. STELLA: Self-Evolving LLM Agent for Biomedical Research - arXiv, accessed February 6, 2026, https://arxiv.org/html/2507.02004v1
2. SeifBenayed/Stella: Stella is an AI-agent tool that provides virtual beta testers with distinct personalities and behaviors for comprehensive UX/UI testing. - GitHub, accessed February 6, 2026, https://github.com/SeifBenayed/Stella
3. Norditech-AB/STELLA: Stella - A scalable platform for creating and managing AI agents, accessed February 6, 2026, https://github.com/Norditech-AB/STELLA
4. Strella – the AI-powered Customer Research Platform, accessed February 6,

2026, https://www.strella.io/
5. Using Pointers In Loops - C Programming - Stack Overflow, accessed February 6, 2026, https://stackoverflow.com/questions/79434124/using-pointers-in-loops-c-programming
6. Demystifying evals for AI agents - Anthropic, accessed February 6, 2026, https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
7. Towards the Holographic Characteristic of LLMs for Efficient Short-text Generation - arXiv, accessed February 6, 2026, https://www.arxiv.org/pdf/2601.22546
8. Publications - Vladlen Koltun, accessed February 6, 2026, http://vladlen.info/publications/
9. ROCR Documentation, accessed February 6, 2026, https://rocm.docs.amd.com/_/downloads/ROCR-Runtime/en/docs-7.0.0/epub/
10. usb_host_library.patch - a1exdandy/checkm8-a5 - GitHub, accessed February 6, 2026, https://github.com/a1exdandy/checkm8-a5/blob/master/usb_host_library.patch
11. UC Riverside - eScholarship.org, accessed February 6, 2026, https://escholarship.org/content/qt45m8r2sx/qt45m8r2sx.pdf
12. HSA-Runtime-AMD/include/hsa.h at master - GitHub, accessed February 6, 2026, https://github.com/HSAFoundation/HSA-Runtime-AMD/blob/master/include/hsa.h
13. Best llms.txt implementation platforms for AI-discoverable APIs in January 2026 - Fern, accessed February 6, 2026, https://buildwithfern.com/post/best-llms-txt-implementation-platforms-ai-discoverable-apis
14. What is llms.txt? Why it's important and how to create it for your docs – GitBook Blog, accessed February 6, 2026, https://www.gitbook.com/blog/what-is-llms-txt
15. LLMs.txt Explained | TDS Archive - Medium, accessed February 6, 2026, https://medium.com/data-science/llms-txt-explained-414d5121bcb3
16. Everything You Ever Wanted to Know About AI Agents - Boomi, accessed February 6, 2026, https://boomi.com/blog/everything-you-ever-wanted-to-know-about-ai-agents/
17. The Agentic Trust Framework: Zero Trust Governance for AI Agents - Cloud Security Alliance, accessed February 6, 2026, https://cloudsecurityalliance.org/blog/2026/02/02/the-agentic-trust-framework-zero-trust-governance-for-ai-agents
18. Toward a Safe Internet of Agents - arXiv, accessed February 6, 2026, https://arxiv.org/html/2512.00520v1
19. A Review of Agentic AI in Cybersecurity: Cognitive Autonomy, Ethical Governance, and Quantum-Resilient Defense - PMC, accessed February 6, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12569510/
20. From truth to trust: the impact of blockchain traceability on trust in product authenticity - Pepperdine Digital Commons, accessed February 6, 2026, https://digitalcommons.pepperdine.edu/cgi/viewcontent.cgi?article=2235&context=etd

21. The Hard-Easy Way: A Veteran Founder's Guide to Startup Validation S03E02, accessed February 6, 2026, https://wildfirelabs.io/s03e02
22. Setting up CIS Benchmark Hardening through Qualys for Baseline Configuration Management: Product Security | by Cyberspecs, accessed February 6, 2026, https://cyberspecs.medium.com/cis-benchmark-hardening-a-product-security-engineers-journey-in-payment-technology-80439bbbff0a
23. Improving IoT Device Security Prevention from Zero-Day Attacks - Hackernoon, accessed February 6, 2026, https://hackernoon.com/improving-iot-device-security-prevention-from-zero-day-attacks